

Dataset Analysis and Notebook Summary

Soham Maity - sohammaity.codaholic81@gmail.com

07/29/2026 22:36:18

<https://gemini.google.com/app/5eae42cf2a2d8e4d?hl=en-IN>

Dataset : <https://www.kaggle.com/datasets/danishjmeo/281k-patient-...>

[Here is a summary and explanation of the provided Jupyter Notebook fil...](#)

What are the features does author have taken to train the model and...

[### Feature Selection and Model Training Analysis](#) The excerpt of the...

Numeric features used as predictors: ['age', 'age_normalized', 'bmi',...

[By selecting only those specific 20 numeric features for model training,...](#)

With these features can I implement 8-multiclass classification model? ...

[Yes, you can absolutely implement an 8-class multiclass classification...](#)

Feature Categories The dataset includes: Demographic information (ag...

[Selecting the optimal features for your 8-class multiclass classification...](#)

Why should I drop these features : high_blood_pressure, blood_pressure...

[You should drop `high\\_blood\\_pressure`, `blood\\_pressure`,...](#)



multidisease-patient-data-analysis-prediction.ipynb

Dataset :

<https://www.kaggle.com/datasets/danishjmeo/281k-patient-dataset-for-multi-disease-prediction>

Please clearly and properly summarize/explain the uploaded ipynb notebook file for this dataset.

7/14/2026, 4:11:49 PM



Here is a summary and explanation of the provided Jupyter Notebook file:

Notebook Overview

The uploaded notebook is titled "Multidisease Patient Data Analysis & Prediction"[cite: 1]. Its stated purpose is to analyze a multidisease patient dataset to discover potential predictors for various disease states[cite: 1]. The notebook was created with assistance from Devra AI[cite: 1].

While the notebook outlines a comprehensive six-step workflow in its Table of Contents, the provided code cells only cover the first three steps: initialization, data loading, and initial exploratory data analysis[cite: 1].

Step-by-Step Breakdown

1. Imports and Helpers

- The notebook begins by suppressing warnings to keep the output clean and sets up `matplotlib` for inline plotting[cite: 1].
- It imports essential data science and machine learning libraries, including `pandas` , `numpy` , `seaborn` , `matplotlib.pyplot` , and modules from `sklearn` (like `LogisticRegression` and `train_test_split`) [cite: 1].

- A custom helper function named `resolve_data_file` is defined to dynamically search for and locate the necessary dataset files within standard Kaggle input directories[cite: 1].

2. Data Loading

- Using the helper function, the notebook successfully locates and loads a file named `Data Warehouse Multiclass.csv` into a pandas DataFrame[cite: 1].
- The loaded dataset is large, consisting of **280,985 rows** (representing individual patient records) and **39 columns** (representing different patient features and labels)[cite: 1].
- A preview of the first few rows reveals a mix of demographic, lifestyle, and medical data, including features like `composite_key`, `age`, `gender`, `bmi`, `smoking`, `hypertension`, `salt_intake`, and `education_level` [cite: 1].

3. Exploratory Data Analysis (EDA)

- The notebook examines the structural information of the dataset, showing that it utilizes approximately 83.6 MB of memory[cite: 1].
- The data consists of 19 numerical columns (`float64`), 1 integer column (`int64`), and 19 categorical/text columns (`object`)[cite: 1].
- Basic statistical summaries are generated for the numerical features, revealing insights into the patient population[cite: 1]:
 - The average age of the patients is approximately **49.6 years**, ranging from 2 to 89 years old[cite: 1].
 - The average BMI is **27.49**, with a minimum of 10.01 and a maximum of 95.69[cite: 1].
 - The mean heart rate across the dataset is about **74.5 beats per minute**[cite: 1].
 - Other health metrics summarize average levels for glucose (135.25), cholesterol (224.04), and systolic blood pressure (134.4)[cite: 1].

(Note: The provided file content concludes immediately after generating the statistical summary of the dataset. The subsequent steps mentioned in the Table of Contents—

Data Cleaning, Modeling and Prediction, and Conclusion—are not present in the provided text[cite: 1].)

7/14/2026, 4:11:49 PM



What are the features does author have taken to train the model and why? And why the other features have been dropped?

7/14/2026, 4:16:25 PM



Feature Selection and Model Training Analysis

The excerpt of the `multidisease-patient-data-analysis-prediction.ipynb` file currently provided does not contain the information necessary to determine which features the author chose to train the model or why specific features were dropped.

Here is why that analysis cannot be completed with the current text:

- **Missing Code Cells:** The uploaded notebook text ends abruptly after generating the statistical summary during the Exploratory Data Analysis (EDA) step[cite: 1].
- **Absent Sections:** The critical code blocks corresponding to "Data Cleaning and Preprocessing" and "Modeling and Prediction"—as outlined in the notebook's Table of Contents—are entirely absent[cite: 1].

Without the specific code demonstrating the preprocessing phase and the model training logic, only the initial 39 raw columns loaded into the dataset are visible[cite: 1].

Could you please provide the remainder of the notebook or the specific code cells covering the data cleaning and modeling steps so the author's feature selection process can be thoroughly analyzed?

7/14/2026, 4:16:25 PM



Numeric features used as predictors:

```
['age', 'age_normalized', 'bmi', 'hypertension',  
'heart_disease', 'HbA1c_level', 'glucose', 'cholesterol',  
'sleep_hours', 'triglycerides', 'blood_pressure',  
'crp_level', 'homocysteine_level', 'systolic_bp',  
'diastolic_bp', 'alcohol_intake', 'salt_intake', 'heart_rate',  
'hdl', 'ldl']
```

Author have considered only these features for model training.

7/14/2026, 4:18:53 PM



By selecting only those specific 20 numeric features for model training, the author has adopted a classic machine learning strategy: prioritizing highly granular, continuous clinical measurements while stripping away redundant categorical data and non-predictive identifiers.

Here is a detailed breakdown of why these specific features were retained and why the remaining 19 columns were dropped.

Why the Numeric Features Were Kept

The selected features (`age` , `bmi` , `HbA1c_level` , `glucose` , `cholesterol` , `systolic_bp` , `crp_level` , etc.) represent the core physiological and demographic metrics of a patient.

- **Clinical Precision:** Continuous numerical values carry exact, granular information. A specific `bmi` of **22.55** or a `glucose` level of **138.0** provides a model with a much higher mathematical signal than a broad bucket like "Normal Weight" or "Pre-diabetic."
- **Direct Correlation to Multidiseases:** Metrics like `homocysteine_level` , `crp_level` (C-reactive protein), `ldl` / `hdl` , and `HbA1c_level` are primary

biomarkers for cardiovascular and metabolic diseases. They are the strongest predictors for the problem the notebook aims to solve.

- **Algorithm Compatibility:** Machine learning algorithms (like the `LogisticRegression` imported in the notebook) compute weights mathematically. Numeric features can be scaled and fed directly into these equations without requiring complex transformations.

Why the Other Features Were Dropped

Out of the original 39 columns, 19 were excluded. These dropped features fall into four specific categories based on standard data science practices:

1. Redundant / Derived Categoricals

Several categorical columns in the dataset are simply text-based representations of the numerical columns that were kept. Including both would introduce **multicollinearity** (where features are highly correlated with each other), which can confuse the model and skew feature importance.

- **Dropped:** `age_level` (Kept: `age`)
- **Dropped:** `bmi_level` (Kept: `bmi`)
- **Dropped:** `high_blood_pressure` (Kept: `systolic_bp` , `diastolic_bp`)
- **Dropped:** `high_ldl_cholesterol` (Kept: `ldl`)
- **Dropped:** `low_hdl_cholesterol` (Kept: `hdl`)

2. Target Variables & Data Leakage Risks

Features that represent the "answer" the model is trying to predict must be removed during training to prevent the model from "cheating" (data leakage).

- **Dropped:** `label` , `sublabel` , and `disease_flags` (These are the actual multi-disease classifications the model is designed to predict).
- **Dropped:** `source_dataset` (If the dataset was combined from different sources, e.g., a "diabetes dataset" and a "heart disease dataset", this column acts as a dead giveaway for the patient's diagnosis).
- **Dropped:** `diabetes` (Likely acts as a target label or leaks the diagnosis directly).

3. Identifiers & Metadata

Machine learning models look for mathematical patterns. Identifiers have no mathematical relationship to health outcomes and act as noise.

- **Dropped:** `composite_key` (A unique string like *Young_Female_Normal_Never_No* provides no numerical value and would crash algorithms that expect numbers).

4. Unencoded Categoricals (Socioeconomic & Lifestyle)

Features like `gender` , `smoking` , `education_level` , `employment_status` , `physical_activity` , `family_history` , `stress_level` , and `sugar_consumption` are valuable, but they are categorical (text).

- **Why they were dropped:** To use these, the author would have needed to perform One-Hot Encoding or Label Encoding (converting "Primary" education into binary columns of 0s and 1s). The author likely dropped them to build a **baseline model** focused purely on clinical lab results, avoiding the dimensionality expansion that comes with encoding text features.

7/14/2026, 4:18:53 PM



With these features can I implement 8-multiclass classification model? or some features I have to add or remove from the actual set of features?

7/14/2026, 5:16:33 PM



Yes, you can absolutely implement an 8-class multiclass classification model using just these 20 numeric features. These continuous variables provide a mathematically rich foundation of cardiovascular and metabolic biomarkers that algorithms can easily process.

However, to maximize your model's predictive accuracy and streamline its eventual deployment, you should optimize this feature set by removing redundancies and reintroducing a few critical categorical features.

Features to Remove (Redundancy & Leakage)

Before training, you should drop features that provide duplicate information or risk compromising the model:

- **age_normalized** : Since the raw **age** column is already in the dataset, including a normalized version introduces perfect multicollinearity. You only need one.
- **blood_pressure** : You are already tracking granular pressure metrics via **systolic_bp** and **diastolic_bp** . A third aggregate blood pressure feature is redundant and adds unnecessary noise.
- **hypertension** and **heart_disease** (**Data Leakage Risk**): You must carefully evaluate your 8 target classes. If "Hypertension" or "Heart Disease" are among the actual diseases your multiclass model is trying to predict, keeping these binary columns in your training data will cause massive data leakage. They should only be kept if they strictly represent *prior* medical history distinct from the target predictions.

Features to Add (Categorical & Clinical Context)

While numeric lab results are excellent predictors, relying *solely* on them leaves out vital clinical context. When engineering end-to-end data science applications, omitting key categorical inputs often leads to flawed inference in a production environment.

You should consider bringing the following features back into your training set:

- **Specific Clinical Symptoms:** If the broader data warehouse contains granular symptom data—such as a specific chest pain classification type—it is highly recommended to include it. Missing specific patient data categories like this during full-stack debugging is a common bottleneck that severely limits a web application's inference capabilities.
- **gender** : Physiological baselines for metrics like **hdl** , **ldl** , and **crp_level** vary significantly across biological sexes.
- **smoking and family_history** : These are primary risk multipliers for multidisease prediction that numeric vitals alone cannot capture until a disease has already progressed.

Handling the Additions Efficiently

Adding categorical features back into the mix does not mean you have to clutter your pipeline with complex One-Hot Encoding.

By implementing an advanced gradient boosting framework like **CatBoost**, you can pass these categorical text features directly into the model. This makes the data pipeline incredibly clean, which is especially useful when passing patient payloads from a Flask web interface to the backend. Alternatively, if you prefer deep learning for this 8-class problem, a Keras sequential model utilizing just a single dense layer can serve as a highly efficient and lightweight architectural schematic for tabular data of this exact scale.

Are you planning to deploy this model as a web application where users will input these specific physiological metrics, or is this strictly for exploratory analysis?

7/14/2026, 5:16:33 PM



Feature Categories

The dataset includes:

Demographic information (age, gender, age group)

BMI and anthropometric measurements

Lifestyle factors (smoking, physical activity, stress, sleep, alcohol, salt, and sugar intake)

Clinical measurements (glucose, HbA1c, cholesterol, triglycerides, HDL, LDL, blood pressure, heart rate, CRP, and homocysteine)

Disease indicators and classification targets

Column Details

composite_key: Unique patient profile identifier.

age_level: Age group (Young, Middle, Mature, Senior).

gender: Patient gender.

bmi_level: BMI category based on WHO standards.

smoking: Smoking status.

diabetes: Diabetes indicator.

age: Patient age in years.

age_normalized: Normalized age (0-1).

bmi: Body Mass Index (kg/m²).

hypertension: Hypertension indicator.

heart_disease: Heart disease indicator.

HbA1c_level: Long-term blood glucose indicator.

glucose: Blood glucose level.

cholesterol: Total cholesterol level.

sleep_hours: Average daily sleep duration.

triglycerides: Triglyceride level.

physical_activity: Physical activity level.

family_history: Family history of chronic disease.

stress_level: Stress level.

low_hdl_cholesterol: Low HDL cholesterol indicator.

high_ldl_cholesterol: High LDL cholesterol indicator.

blood_pressure: Blood pressure measurement.

high_blood_pressure: High blood pressure indicator.

sugar_consumption: Daily sugar intake level.
crp_level: C-Reactive Protein (CRP) level.
homocysteine_level: Homocysteine level.
systolic_bp: Systolic blood pressure.
diastolic_bp: Diastolic blood pressure.
alcohol_intake: Alcohol consumption level.
salt_intake: Dietary salt intake level.
heart_rate: Resting heart rate.
hdl: HDL cholesterol level.
ldl: LDL cholesterol level.
education_level: Education level.
employment_status: Employment status.
source_dataset: Original source dataset.
disease_flags: Disease combination code.
sublabel: Multi-class disease label (8 classes), they are
- Normal, Diabetes, Diabetes-Hypertension, Diabetes-HeartDisease, Hypertension, Hypertension-HeartDisease, HeartDisease, Diabetes-Hypertension-HeartDisease.
label: Binary target label (Normal or Abnormal).

Which features should I considered for multiclass classification and which I should not? also explain why?

7/14/2026, 5:28:49 PM



Selecting the optimal features for your 8-class multiclass classification model requires balancing maximum clinical signal with the strict avoidance of data leakage and mathematical redundancy.

Your target variable for this task is **sublabel** , which contains the eight distinct combinations of Normal, Diabetes, Hypertension, and Heart Disease. Every other feature must be evaluated strictly on its ability to predict this outcome without "cheating" or confusing the algorithm.

Features to Retain

You should keep the foundational, highly granular data points that capture a patient's true physiological state, behavioral risks, and demographic baseline.

Feature Category	Selected Features	Rationale for Inclusion
Clinical Biomarkers	HbA1c_level , glucose , cholesterol , triglycerides , crp_level , homocysteine_level , systolic_bp , diastolic_bp , heart_rate , hdl , ldl	These raw, continuous variables form the mathematical core of cardiovascular and metabolic predictions. They provide highly specific signals that algorithms can easily weigh.
Lifestyle & History	bmi , smoking , sleep_hours , physical_activity , family_history , stress_level , sugar_consumption , alcohol_intake , salt_intake	These capture crucial behavioral risk multipliers that often precede clinical biomarker degradation, giving the model predictive power for early-stage disease risks.
Demographics	age , gender , education_level , employment_status	age and gender establish critical physiological baselines. Socioeconomic indicators like education and employment status are strong proxy variables for healthcare access and long-term health outcomes.

Features to Drop

More than a third of the dataset must be discarded to build a robust, production-ready model. Including these features will actively harm the model's validity or performance.

Exclusion Reason	Dropped Features	Rationale for Exclusion
Severe Data Leakage	label , disease_flags , diabetes , hypertension , heart_disease , source_dataset	This is the most critical exclusion. The individual disease indicators (diabetes , hypertension , heart_disease) are the literal

Exclusion Reason	Dropped Features	Rationale for Exclusion
		components of your 8-class <code>sublabel1</code> . Keeping them allows the model to "cheat" and perfectly memorize the answer rather than learning to predict it.
Mathematical Redundancy	<code>age_level</code> , <code>age_normalized</code> , <code>bmi_level</code> , <code>high_blood_pressure</code> , <code>blood_pressure</code> , <code>low_hdl_cholesterol</code> , <code>high_ldl_cholesterol</code>	These are simply bucketed or derived versions of the raw numerical columns you are already keeping. Passing both the granular number and the broad category into a model causes multicollinearity, which distorts feature importance.
Non-Predictive Noise	<code>composite_key</code>	This is a unique identifier string with zero predictive value. Retaining it will bloat the dataset and crash algorithms that expect numerical matrices.

Schema Gaps and Deployment Considerations

Critically evaluating this specific schema reveals a missing patient data category: there is no chest pain classification type input. Omitting this specific symptom data often creates a significant predictive blind spot and subsequent full-stack debugging issues when transitioning models from a notebook into a live production environment.

When you move to deploy this pipeline—for instance, serving predictions via a Flask backend—the retained mix of categorical and numerical features must be handled cleanly. Utilizing an advanced gradient boosting framework like CatBoost allows you to pass the categorical text features (like `employment_status` or `smoking`) directly into the model without requiring cumbersome one-hot encoding pipelines. Alternatively, if your architectural schematic relies on neural networks, a Keras sequential model defined with only one dense layer provides a highly efficient, computationally light approach for tabular data of this specific scale.

Are you planning to rely on native algorithm support for the categorical features, or do you have a specific encoding and scaling pipeline in mind for the lifestyle and demographic inputs?



Why should I drop these features :
high_blood_pressure, blood_pressure,
low_hdl_cholesterol, high_ldl_cholesterol?

7/14/2026, 5:34:24 PM



You should drop `high_blood_pressure` , `blood_pressure` , `low_hdl_cholesterol` , and `high_ldl_cholesterol` because they suffer from **mathematical redundancy**. In data science, this is known as introducing **multicollinearity** into your dataset.

Here is a straightforward breakdown of exactly why keeping them actively works against your model.

1. They Are Derived Duplicates

Machine learning models thrive on unique mathematical variance. These four features do not provide any *new* information; they are simply text-based or simplified derivations of the granular numerical features you are already keeping.

- **The Blood Pressure Overlap:** You are already retaining `systolic_bp` and `diastolic_bp` , which give the exact millimeter of mercury (mmHg) readings of a patient's blood pressure. The `high_blood_pressure` column is just a binary threshold applied to those exact numbers (e.g., if `systolic` > 130, then "Yes"). Similarly, `blood_pressure` is likely just a categorical grouping of those numbers.
- **The Cholesterol Overlap:** You are already retaining `hdl` and `ldl` as continuous numbers. The `low_hdl_cholesterol` and `high_ldl_cholesterol` columns are just "Yes/No" flags triggered when the raw `hdl` or `ldl` numbers cross a specific medical threshold.

2. How This Harms the Model

If you feed both the exact number and the derived category into your algorithm, it causes several problems:

- **Confused Feature Importance:** When two features represent the exact same thing, the algorithm struggles to figure out which one is actually driving the prediction. It splits the "importance" weight between them, making it much harder for you to interpret which features are truly causing the disease.
- **Risk of Overfitting:** Feeding the model the same information twice essentially gives that specific health metric "double votes" during training. The model might over-fixate on blood pressure simply because there are four columns talking about it, rather than learning the balanced relationship between all vitals.
- **Loss of Nuance (Information Destruction):** Continuous numbers contain a gradient of risk. An `ldl` of 159 is worse than an `ldl` of 105. If you rely on a binary `high_ldl_cholesterol` flag, the model just sees "Yes" for both patients and loses the ability to measure the severity. By keeping the raw numbers and dropping the binary flags, you give the model the maximum amount of detail without any of the noise.

By dropping these categorical derivations, you are forcing the model to learn the mathematical thresholds on its own using the pure, raw clinical data.

Since you are relying on the continuous, raw numbers like `systolic_bp` and `ldl`, are you planning to apply any feature scaling (like Standard Scaler or MinMax Scaler) to ensure features with larger numeric ranges do not dominate the algorithm?

7/14/2026, 5:34:24 PM